Book Reading List Using Linked List

1. Title

Book Reading List Using Linked List

2. Problem Statement

Develop a **Book Reading List Management System using a Singly Linked List in C++**. The system should allow the user to add, display, search, delete, and count books in the reading list. Each book should be stored in a separate node containing the book name and a pointer to the next node.

3. Objective

- To understand the concept of a **Singly Linked List**.
- To create and manage nodes dynamically.
- To add books to the reading list.
- To display all books in the list.
- To search for a particular book.
- To delete a book from the list.
- To count the total number of books.
- To understand dynamic memory allocation in C++.

4. Algorithm

Step 1:

Start the program.

Step 2:

Create a Node structure containing:

- bookName
- next pointer

Step 3:

Initialize head as NULL.

Step 4:

Display the menu:

1. Add Book

2. Display Books
3. Search Book
4. Delete Book
5. Count Books
6. Exit

Step 5:

If the user selects **Add Book**:

- Create a new node.
- Enter the book name.
- If the list is empty, make the new node the head.
- Otherwise, traverse to the last node and connect the new node.

Step 6:

If the user selects **Display Books**:

- Start from head.
- Display each book.
- Move to the next node until NULL is reached.

Step 7:

If the user selects **Search Book**:

- Enter the book name.
- Compare it with each node.
- Display "Book found" if a match is found.
- Otherwise display "Book not found".

Step 8:

If the user selects **Delete Book**:

- Enter the book name.
- Search for the book.
- Change the links to remove the node.
- Delete the node from memory.

Step 9:

If the user selects **Count Books**:

- Traverse the linked list.
- Increase the counter for every node.
- Display the total number of books.

Step 10:

If the user selects **Exit**, terminate the program.

Step 11:

Stop.

5. Conclusion

The **Book Reading List Using Linked List** program successfully demonstrates the implementation of a **Singly Linked List in C++**. It performs important operations such as **insertion, traversal, searching, deletion, and counting**. This project helps in understanding linked lists and dynamic memory allocation through a simple real-life application.